Engineering Report - AIOps Observability Lab 1

1. Objective

Build a Laravel API that emits ML-ready telemetry and exposes RED (Rate, Errors, Duration) metrics for anomaly detection and incident triage. The experiment must inject a controlled anomaly window with machine-readable ground truth.

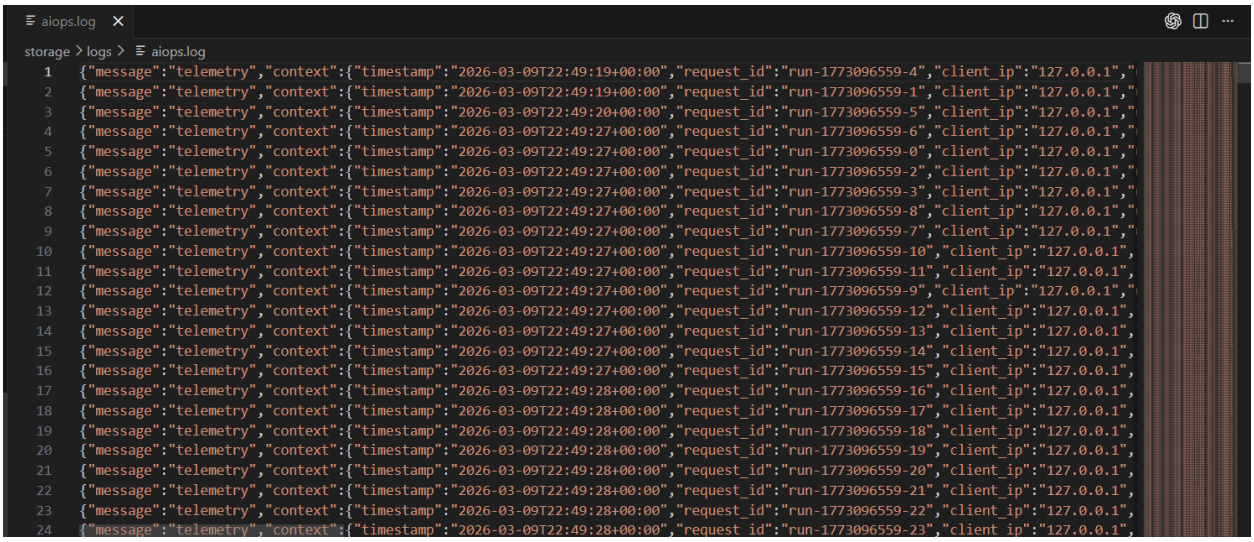
---

2. Telemetry Design

2.1 Log Schema

The telemetry record is intentionally stable and includes the exact same keys for every request to simplify downstream ingestion and model training:

- timestamp: Event time in ISO-8601 for ordering and correlation with metrics.
- request_id: Cross-service correlation key (X-Request-Id propagated/generated).
- client_ip / user_agent: Source context for traffic profiling.
- method / path / query / route_name: Request identity and route-level grouping.
- payload_size_bytes / response_size_bytes: Payload pressure and bandwidth signals.
- status_code / latency_ms: Core RED dimensions for response and performance.
- severity: Quick triage level (info for normal, error for categorized failures).
- error_category: Canonical class used by logs and metrics.
- build_version: Release-aware debugging and regression isolation.
- host: Node identity for multi-instance deployments.



2.2 Error Categorization

Error classification is centralized in app/Exceptions/Handler.php to ensure uniformity across logs and metrics:

Error Category	Exception Source
VALIDATION_ERROR	ValidationException
DATABASE_ERROR	QueryException
SYSTEM_ERROR	Uncaught server-side failures
UNKNOWN	Other 4xx-class failures

Latency-Aware Overrides:

Middleware applies a latency-aware override, flagging a TIMEOUT_ERROR when request latency exceeds 4000 ms, *even if the status code is 200*. This supports the lab requirement for capturing "successful but unhealthy" requests (e.g., /api/slow?hard=1).

2.3 Correlation & Propagation

Middleware reads the X-Request-Id when provided. If absent, it generates a UUID. This ID is always written to the response header, keeping request traces perfectly linkable across logs and downstream clients.

---

3. Metrics Engineering

3.1 RED Metrics

We implemented Prometheus metrics to capture Rate, Errors, and Duration:

Counters:

- http_requests_total{method, path, status}
- http_errors_total{method, path, error_category}

Histograms:

- http_request_duration_seconds_bucket{method, path, le}
- http_request_duration_seconds_sum{method, path}
- http_request_duration_seconds_count{method, path}

Buckets: Configured as [0.05, 0.1, 0.25, 0.5, 1, 2.5, 5, 10, +Inf] to effectively capture:

- Fast responses (/api/normal)
- Moderate delays (/api/slow)
- Severe delays (/api/slow?hard=1)

127.0.0.1:8000/metrics

```
# HELP http_requests_total Total number of HTTP requests processed.
# TYPE http_requests_total counter
http_requests_total{method="GET",path="/",status="200"} 9
http_requests_total{method="GET",path="/",status="500"} 9
http_requests_total{method="GET",path="/api/db",status="200"} 218
http_requests_total{method="GET",path="/api/db",status="500"} 65
http_requests_total{method="GET",path="/api/error",status="500"} 424
http_requests_total{method="GET",path="/api/normal",status="200"} 5926
http_requests_total{method="GET",path="/api/slow",status="200"} 1893
http_requests_total{method="POST",path="/api/validate",status="200"} 98
http_requests_total{method="POST",path="/api/validate",status="302"} 35
http_requests_total{method="POST",path="/api/validate",status="422"} 47
# HELP http_errors_total Total number of HTTP error-classified requests.
# TYPE http_errors_total counter
http_errors_total{method="GET",path="/",error_category="DATABASE_ERROR"} 5
http_errors_total{method="GET",path="/",error_category="TIMEOUT_ERROR"} 4
http_errors_total{method="GET",path="/api/db",error_category="DATABASE_ERROR"} 20
http_errors_total{method="GET",path="/api/db",error_category="TIMEOUT_ERROR"} 45
http_errors_total{method="GET",path="/api/error",error_category="SYSTEM_ERROR"} 424
http_errors_total{method="GET",path="/api/slow",error_category="TIMEOUT_ERROR"} 588
http_errors_total{method="POST",path="/api/validate",error_category="VALIDATION_ERROR"} 82
# HELP http_request_duration_seconds Request duration in seconds.
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{method="GET",path="/",le="+Inf"} 18
http_request_duration_seconds_bucket{method="GET",path="/",le="0.05"} 6
http_request_duration_seconds_bucket{method="GET",path="/",le="0.1"} 8
http_request_duration_seconds_bucket{method="GET",path="/",le="0.25"} 11
http_request_duration_seconds_bucket{method="GET",path="/",le="0.5"} 13
http_request_duration_seconds_bucket{method="GET",path="/",le="1"} 13
http_request_duration_seconds_bucket{method="GET",path="/",le="10"} 17
http_request_duration_seconds_bucket{method="GET",path="/",le="2.5"} 14
http_request_duration_seconds_bucket{method="GET",path="/",le="5"} 14
http_request_duration_seconds_bucket{method="GET",path="/api/db",le="+Inf"} 283
http_request_duration_seconds_bucket{method="GET",path="/api/db",le="0.05"} 221
http_request_duration_seconds_bucket{method="GET",path="/api/db",le="0.1"} 221
http_request_duration_seconds_bucket{method="GET",path="/api/db",le="0.25"} 233
http_request_duration_seconds_bucket{method="GET",path="/api/db",le="0.5"} 238
http_request_duration_seconds_bucket{method="GET",path="/api/db",le="1"} 238
http_request_duration_seconds_bucket{method="GET",path="/api/db",le="10"} 283
http_request_duration_seconds_bucket{method="GET",path="/api/db",le="2.5"} 238
http_request_duration_seconds_bucket{method="GET",path="/api/db",le="5"} 238
http_request_duration_seconds_bucket{method="GET",path="/api/error",le="+Inf"} 424
http_request_duration_seconds_bucket{method="GET",path="/api/error",le="0.05"} 424
http_request_duration_seconds_bucket{method="GET",path="/api/error",le="0.1"} 424
http_request_duration_seconds_bucket{method="GET",path="/api/error",le="0.25"} 424
http_request_duration_seconds_bucket{method="GET",path="/api/error",le="0.5"} 424
http_request_duration_seconds_bucket{method="GET",path="/api/error",le="1"} 424
```

3.2 Cardinality Controls

Metric labels are strictly restricted to low-cardinality fields (method, normalized path, status, error_category). High-cardinality data like request IDs or raw query strings are intentionally excluded from labels to prevent label explosion and memory exhaustion.

3.3 Metrics Exclusion

The /metrics endpoint is excluded from logging and instrumentation to avoid self-generated noise and artificial request-rate inflation.

---

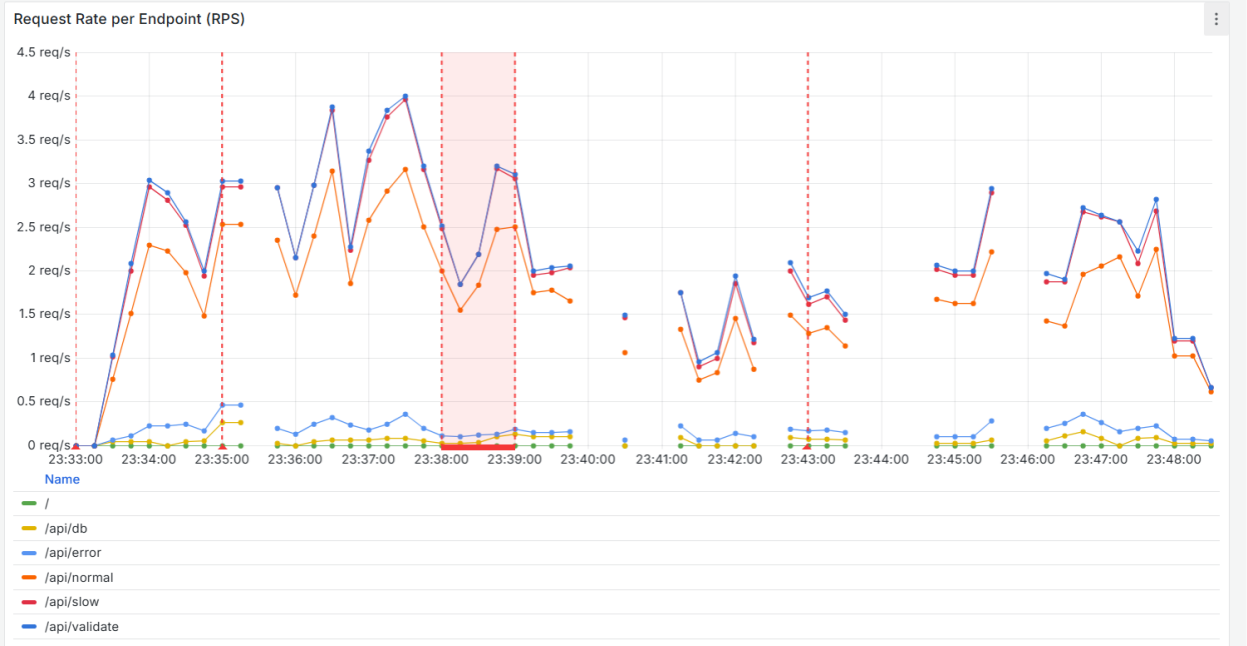
4. Monitoring Stack

The monitoring infrastructure is provisioned via docker-compose.yml, spinning up Prometheus and Grafana.

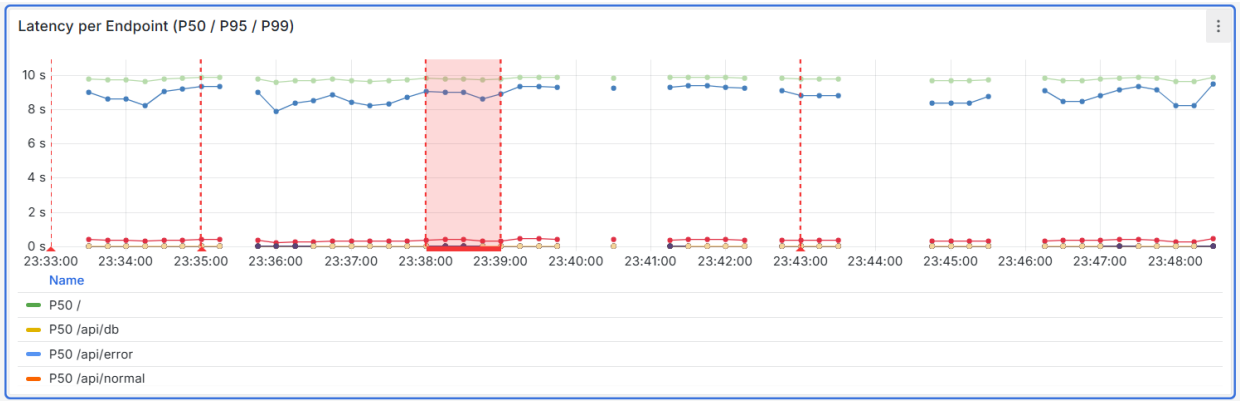
Grafana Dashboard Requirements (Exported):

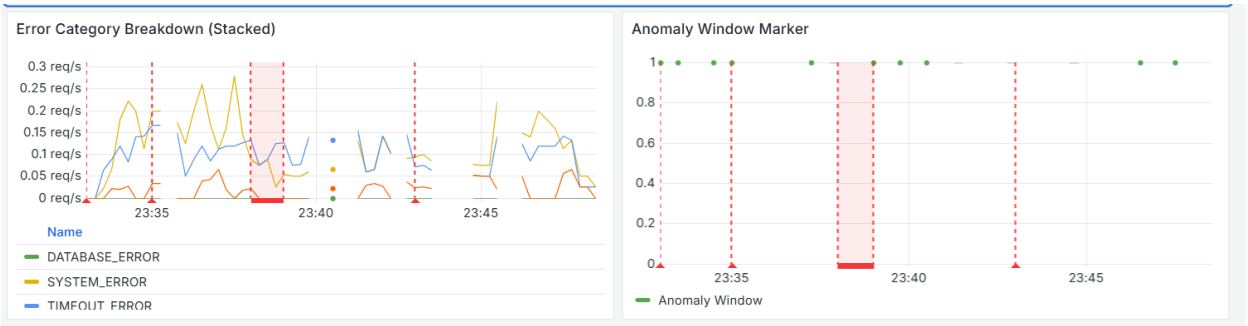
- **RPS:** Requests per second per endpoint.
- **Error Rate:** Percentage of errors per endpoint.
- **Latency Quantiles:** P50, P95, and P99 response times per endpoint.
- **Error Breakdown:** Stacked visualization of error categories.
- **Anomaly Window Marker:** Visual indicator of the injected anomaly.

Anomaly Marker Signal: Tracks the aiops_anomaly_window gauge (1 = active, 0 = inactive).


Request Rate per Endpoint (RPS)
4.5 req/s
4 req/s
3.5 req/s
3 req/s
2.5 req/s
2 req/s
1.5 req/s
1 req/s
0.5 req/s
0 req/s
23:33:00
23:34:00
23:35:00
23:36:00
23:37:00
23:38:00
23:39:00
23:40:00
23:41:00
23:42:00
23:43:00
23:44:00
23:45:00
23:46:00
23:47:00
23:48:00
Name
/
/api/db
/api/error
/api/normal
/api/slow
/api/validate

Error Rate % per Endpoint
100%
80%
60%
40%
20%
0%
23:35
23:40
23:45
Name
/
/api/db
/api/error


Latency per Endpoint (P50 / P95 / P99)
10 s
8 s
6 s
4 s
2 s
0 s
23:33:00
23:34:00
23:35:00
23:36:00
23:37:00
23:38:00
23:39:00
23:40:00
23:41:00
23:42:00
23:43:00
23:44:00
23:45:00
23:46:00
23:47:00
23:48:00
Name
P50 /
P50 /api/db
P50 /api/error
P50 /api/normal



---

5. Controlled Anomaly Experiment

Traffic Generator (traffic_generator.py):

- Applies a base load for 8–12 minutes.
- Injects an exact 2-minute anomaly window.
- Uses a distribution aligned with the lab specification.

Generated Artifacts:

1. ground_truth.json
2. logs.json
3. traffic_run_summary.json

Ground Truth Fields Collected:

- anomaly_start_iso
- anomaly_end_iso
- anomaly_type
- expected_behavior

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\am524\OneDrive\Desktop\College\Semester 8\Selected Topics\Labs\Lab1\Lab1_Ahmed> python tr
affic_generator.py --base-url http://127.0.0.1:8000 --base-minutes 8 --min-base-requests 3000 --anoma
ly-type latency_spike --max-workers 16 --timeout-seconds 60 --reset-aiops-log --output-dir .
>>
[2026-03-09T22:49:19+00:00] reset log file: C:\Users\am524\OneDrive\Desktop\College\Semester 8\Select
ed Topics\Labs\Lab1\Lab1_Ahmed\storage\logs\aiops.log
[2026-03-09T22:49:19+00:00] experiment_start
[2026-03-09T22:57:19.826958+00:00] anomaly_start (latency_spike)
[2026-03-09T22:59:19.826958+00:00] anomaly_end
```

---

6. Why This Is ML-Ready

This implementation serves as a robust foundation for machine learning models due to the following structural guarantees:

- **Stable Schema:** Explicit error taxonomy with predictable keys.
- **Traceability:** Correlation IDs allow for reliable cross-event stitching.
- **Contextual Health:** Latency-aware timeout classification operates independently of standard HTTP status codes.
- **Quantile-Ready:** RED metrics are structured with optimized histogram bucket designs.
- **Supervised Learning Ready:** The distinctly labeled anomaly window provides machine-readable ground truth for training anomaly detection and incident triage pipelines.

---

7. Validation Checklist

- [x] Endpoint coverage includes normal, slow, error, random, db, and validate.
- [x] Validation and DB failures are explicitly separated (VALIDATION_ERROR vs DATABASE_ERROR).
- [x] Timeout-over-200 behavior is successfully captured (TIMEOUT_ERROR with status_code=200).
- [x] Prometheus metrics accurately expose the required counters and histograms.
- [x] Grafana dashboard features all required visibility panels (RPS, Latency, Errors, Anomaly marker).

- [x] Traffic generator successfully exports ground truth and the structured logs dataset.